D I S C O V E R Y

Guest Profiling System Discovery Report

Step 1 of 5 — Reference Repositories for Runtime-Behavior Reverse Engineering

Project: Guest Profiling System · Repo: lordwhitefire/guest-profiling-system

Phase: 1 of 5 — Discovery

Prepared by: Z.ai Builder Agent

Date: 2026-11-10

1. Executive Summary

This report opens the build phase of the Guest Profiling System by identifying five open-source repositories whose runtime behavior is relevant to the project. The goal of the project is a hotel add-on that reads existing guest records (read-only) from a hotel's Property Management System (PMS), presents live-filtered matching profiles as a receptionist types, and auto-fills the hotel's existing registration form on click. The system never writes back to the hotel's database and never replaces the PMS.

The project's distinctive architectural choice is a one-way data flow: the hotel's existing database (referred to as the messy source) is read periodically and used to update our own clean, profile-shaped database. Our schema is structured for fast guest lookup; the source schema is whatever the hotel's PMS vendor shipped. This makes the adapter a one-way synchronization problem, not a query-translation problem.

Five repositories were selected after parallel searches across five behavior domains: hotel PMS software, hospitality guest CRM, fuzzy guest search, browser form auto-fill, and read-only database synchronization. Together they cover every behavioral surface the Guest Profiling System must reproduce at runtime. Per the project methodology, no source code from these repositories will be reused; only their settled runtime behavior — input shapes, output shapes, latency characteristics, ranking rules, edge cases, and error behavior — will be captured as stack-agnostic behavioral specification cards in Step #2.

The single most valuable discovery is erkkah/sslr, a small Go utility that performs read-only, throttled, filtered replication from a PostgreSQL source using only a read-only connection — exactly the contract the PMS Adapter must satisfy when the hotel grants us a read-only database login. The single most instructive discovery is aws-samples/sample-hospitality-systems, which publishes the cleanest guest API contract (POST/GET/PUT /guests, paginated list, linked reservations, linked loyalty, multi-tenant isolation) and gives us a strong reference for what our own profile API should look like at runtime.

2. Background and Objectives

2.1 Why this report exists

The Guest Profiling System repository at lordwhitefire/guest-profiling-system contains a portable context bundle (agent definitions, skill files, and a plain-English plan) but no production code. Before any reverse-engineering, planning, or build step can begin, we

need a defensible shortlist of open-source systems that already exhibit the behaviors we must reproduce. This report is that shortlist, with a one-paragraph runtime-behavior breakdown for each candidate and a cross-cutting pattern analysis at the end.

2.2 What the report does NOT cover

This report does not reverse-engineer any repository. Reverse-engineering happens in Step #2, after this shortlist is reviewed. This report does not propose a stack, schema, or architecture — those decisions belong to the Product-Manager planning step (Step #3) and the OpenSpec design step (Step #4). This report does not evaluate proprietary PMS systems such as Opera, Cloudbeds, Mews, or Apaleo beyond their publicly documented APIs; runtime behavior of closed-source software is out of scope.

2.3 Project constraints carried into discovery

The following hard constraints, taken from the project plan and the conversation that opened this build, shaped the discovery criteria:

- Read-only access to the hotel's PMS data. The system never writes to the source database.
- One-way data flow: source DB (messy) → our DB (clean, profile-shaped). Our schema, their data.
- Single-developer maintainable. Per-hotel adapter pattern; shared core across all hotels.
- NDPR (Nigeria), GDPR (EU), and CCPA (US) compliance for guest personal data.
- No AI features, no automatic duplicate detection, no spell correction (per the project plan).
- Frontend is designed from scratch with an art-directed, polished, modern interface; no frontend reverse-engineering.

3. Scope, Data Sources and Methodology

3.1 Search strategy

Five parallel web searches were executed against the GitHub and general web corpus, one per behavior domain: open-source hotel PMS, hospitality guest CRM, fast fuzzy guest search, browser form auto-fill, and read-only PostgreSQL ETL/sync. The top results from each search were fetched and inspected. From the resulting pool, the single strongest

candidate per domain was selected for the shortlist, plus one additional PMS candidate that already exhibits a clean guest API.

3.2 Selection criteria

A candidate was admitted to the shortlist only if it satisfied all of the following:

- Open-source license (or explicit public AWS sample license) so behavioral study is legal.
- Actively maintained or at minimum publicly archived on GitHub with a readable README.
- Exhibits at least one runtime behavior the Guest Profiling System must reproduce.
- Behavior is observable from outside (public API, demo, documented endpoints, or readable runtime contract).
- Distinct from the other selected candidates — no two repos cover the same behavioral surface.

3.3 Method limitation

Discovery was performed entirely from public web sources: GitHub repository pages, README files, and documentation pages reachable without authentication. No repository was cloned, no binary was executed, and no proprietary software was inspected. The behavioral descriptions in Section 4 are therefore limited to what is publicly documented; deeper runtime behavior will be captured in Step #2 by cloning the open-source repositories and observing their actual runtime responses.

4. Core Findings — Five Reference Repositories

The five repositories below are listed in the order they will be useful during Step #2 reverse-engineering. Each entry states the repository, its observable runtime behavior, why it matters to this project, and what specifically should be captured in Step #2.

4.1 QloApps — Open-source hotel PMS (the messy source)

Repository: github.com/qloapps/qloapps · License: OSL-3.0 (core) / AFL-3.0 (modules) · Stack: PHP 8.2+ / MySQL 5.7+ / Apache or Nginx

QloApps is a complete open-source hotel management and reservation system. It is the closest open-source analog to the kind of legacy PMS our adapter must read from. Its runtime behavior is that of a classic PHP server-rendered application: guests are persisted

in MySQL tables tied to bookings, room inventory, and order history; guest search happens via the application layer over those tables; a webservice folder exposes API endpoints consumed by both the back office and the booking engine.

Why it matters: it gives us a real-world example of a messy source PMS schema. The exact table names and column names are not yet documented on the repository landing page, but the project structure (PHP classes for guests, bookings, rooms, orders) is the shape of data our adapter will have to ingest. Studying how QloApps models guests, what fields it tracks, how it links guests to multiple bookings over time, and how it handles returning-guest lookup at the registration screen is the behavioral input for our schema-mapping layer.

What to capture in Step #2: the runtime contract of a hotel PMS guest record. Concretely — what fields a guest row carries, how a guest is linked to booking history, how the existing check-in screen looks up a returning guest, what the webservice API exposes, and what the timing characteristics of guest search look like on a representative dataset.

4.2 AWS Sample Hospitality Systems — Cloud-native PMS with a clean guest API

Repository: github.com/aws-samples/sample-hospitality-systems · License: AWS Sample Code · Stack: Python 3.12 Lambda / Aurora Serverless v2 PostgreSQL / Cognito / React 18 + Vite + Tailwind

This AWS sample ships a full serverless hospitality platform split into two surfaces: a public Central Reservation System (CRS) and a staff-facing PMS. Both expose REST APIs through API Gateway, with all writes going to Aurora PostgreSQL via RDS Proxy using IAM authentication. The runtime behavior of the guest API is the cleanest publicly documented contract found during discovery.

Table 1: AWS Sample Hospitality — observed guest API runtime contract

Method	Endpoint	Runtime behavior
POST	/guests	Creates a new guest record. Returns the new guestId. Auth: Cognito ID token (Bearer).
GET	/guests/{guestId}	Returns full guest details. Auth: Cognito ID token. Tenant-isolated.
PUT	/guests/{guestId}	Updates guest details. Auth: Cognito ID token. Tenant-isolated.
GET	/guests/{guestId}/reservations	Returns the guest's reservation history. Linked resource.

Method	Endpoint	Runtime behavior
GET	/guests/{guestId}/loyalty	Returns the guest's loyalty profile (points, tier). Linked resource.
GET	/guests?page=&limit=	Paginated guest list. PMS-only (staff-facing). Tenant-isolated.

Why it matters: this is the closest thing to a reference architecture for the clean side of our system. It demonstrates three runtime behaviors we must reproduce. First, multi-tenant isolation: every PMS handler calls a shared tenant.py utility so a staff member can only see guests associated with their property or region. Second, linked resources: a guest record is the hub; reservations and loyalty are spokes returned via dedicated sub-resources rather than embedded in the guest payload. Third, split audience: the guest-facing CRS can create and read guests, but only the staff-facing PMS can list them.

What to capture in Step #2: the exact request/response JSON shapes (which we can clone locally and exercise), the tenant-isolation enforcement pattern, the pagination contract (page/limit semantics, total-count header presence, next-cursor pattern if any), the JWT/Cognito claim shape, and the latency profile of each endpoint under cold and warm Lambda conditions.

4.3 KevAlien CRM — Hospitality guest journey state machine

Repository: github.com/KevAlien/CRM · License: not specified in README · Stack: Python + LangGraph + FastAPI + Redis + Ollama (local LLM)

KevAlien CRM is a hospitality-focused guest relationship manager. It identifies guests by their WhatsApp contact information, persists each guest's full journey in a Redis-backed GuestState, and walks each guest through a deterministic state machine. The WhatsApp channel and the local-LLM orchestration are out of scope for this project, but the underlying guest-state model is directly useful.

> *UNKNOWN_CONTACT → ACQUIRING → BOOKING_RECEIVED → WELCOME_SENT → INFO_SENT → IDLE → UPSELL → PRE_CHECKIN → CHECKIN_DAY → IN_HOUSE → POST_STAY. Any phase → ESCALATED (on complaint or staff request). ESCALATED → IDLE (after staff resolves and reactivates).*

Why it matters: our profile store needs to know where a guest is in their lifecycle so the receptionist screen can surface relevant context (e.g., arriving today, in-house, repeat guest with three prior stays). The KevAlien state machine is the most compact, hospitality-specific enumeration of those phases encountered during discovery. We will not reproduce the

WhatsApp or AI parts; we will reproduce the phase vocabulary and the persistence pattern (Redis primary, in-memory fallback).

What to capture in Step #2: the full state-machine transition table (allowed transitions only), the GuestState TypedDict field list (whatever is publicly visible in graph/state.py), the Redis key structure for guest sessions, and the graceful-degradation behavior when Redis is unavailable.

4.4 erkkah/sslr — Read-only PostgreSQL replication (the sync primitive)

Repository: github.com/erkkah/sslr · License: not specified in README (verify before any non-behavioral use) · Stack: Go

SSLR (Simple Stupid Logical Replication) is a small Go utility that keeps a target PostgreSQL database synchronized with a source PostgreSQL database using only a read-only connection. It requires no server-side extensions, no logical replication slot, and no superuser privileges on the source. This is precisely the contract the PMS Adapter must satisfy when a hotel grants us a read-only database login against their PMS database.

Table 2: SSLR — observed runtime replication behavior

Behavior	Runtime characteristic
Connection	Single read-only connection to source. No server-side config; no extensions installed.
Change detection — inserts/updates	Tracks per-table xmin values; pulls rows whose xmin exceeds the last synced value.
Change detection — deletes	Compares primary-key hashes between source and target via divide-and-conquer; splits PK ranges until deletions are localized.
State storage	Highest synced xmin is persisted in the target database as the replication cursor.
Throttling	Configurable source-DB utilization cap (percentage); jobs run in chunks.
Filtering	Per-table WHERE clauses; supports dependent-table filters (WHERE referencing other tables).
Run modes	One-shot sync pass (default) or continuous (-c flag). Delete scans can run as a separate, less-frequent job.
Parallelism	Multiple jobs in parallel as long as different tables are handled by each job.

Why it matters: this is the single most directly applicable discovery. The contract our adapter must honor — read-only on source, no server-side config, detect inserts/updates/deletes, throttle to be polite to the production PMS, filter to guest-related

tables only, store progress in our own DB — is exactly SSLR's contract. We are not going to import SSLR's code, but its runtime behavior is the specification our adapter must match.

What to capture in Step #2: the exact chunking algorithm (chunk size selection, throttle percent interpretation), the divide-and-conquer delete-detection algorithm, the state-row schema in the target DB, the configuration file shape (sslr.json), and the failure modes (what happens on partial sync, on connection loss, on schema drift). These are the runtime behaviors we will re-implement in our adapter's TypeScript codebase.

4.5 FormFilla — Smart browser form-fill with scored field matching

Repository: github.com/AhmadHassan-BTed/FormFilla · License: per README (verify) · Stack: TypeScript / Chrome + Edge extension · 4.7k stars

FormFilla is a local-first browser extension that fills job-application forms from local JSON profile data. The reason it appears in this discovery report — despite being job-application-focused rather than hospitality-focused — is that its form-fill runtime behavior is the most sophisticated of any open-source autofill extension encountered. Specifically, it does not rely on naive field-name matching; it uses multi-signal scoring, structural repeated-block detection, and bipartite optimization (the Hungarian algorithm) to assign profile fields to form fields.

Why it matters: when the hotel's PMS is browser-based, our PMS Adapter is a browser extension that must inject guest data into the hotel's existing registration form. Hotel registration forms are not standardized — field names vary by vendor, fields are added and removed over time, and dynamic repeatable blocks (multiple guests per registration) are common. A scoring-based, structure-aware filler is the right runtime contract; naive name-based fillers will break on the first vendor update.

What to capture in Step #2: the multi-signal scoring formula (which signals are combined and with what weights), the structural repeated-block detection algorithm, the Hungarian-algorithm matching step, the JSON profile schema, the trigger model (what event initiates a fill), and the user-facing confirmation flow before values are written into the form.

5. Problem Diagnosis and Cross-Cutting Patterns

5.1 The adapter is two problems, not one

Discovery clarified that the PMS Adapter is not a single component. It is two distinct runtime behaviors that share a name. The first is the read-side sync: pull guest records from

the hotel's existing PMS database into our own profile database, periodically and read-only. SSLR is the behavioral reference. The second is the write-side fill: push guest profile data into the hotel's existing registration form when the receptionist selects a profile. FormFilla is the behavioral reference. Conflating these two into a single component is a likely source of fragility; treating them as separate adapters with a shared guest-data contract is cleaner.

5.2 Multi-tenant isolation is not optional

The AWS sample enforces property and region isolation at every PMS handler via a shared tenant.py utility. Even though the first hotel deployment will be single-tenant, building the isolation boundary in from day one is materially cheaper than retrofitting it. The behavioral contract to reproduce: every guest query carries an implicit tenant filter, and the filter is enforced in a single shared code path, not per-handler.

5.3 Linked resources over fat payloads

Both the AWS sample and KevAlien CRM treat the guest record as a hub with linked spokes (reservations, loyalty, communication history) rather than embedding everything in one payload. The runtime contract to reproduce: GET /guests/{id} returns the canonical guest; reservations, loyalty, and history are fetched on demand via sub-resources. This keeps the live-search payload small (critical for sub-300ms receptionist typing responsiveness) and avoids over-fetching when only a profile match is needed.

5.4 State machine vocabulary is reusable across hotels

KevAlien's guest-state machine (UNKNOWN_CONTACT through POST_STAY with ESCALATED as a side state) is small, hospitality-specific, and vendor-neutral. Adopting it as our internal phase vocabulary means the receptionist screen can render contextual hints ("arriving today", "currently in-house", "checked out 2 days ago") without each adapter having to invent its own phase model. The behavioral contract: phases are derived from reservations in our database, not pushed by the PMS.

5.5 Read-only replication has a known good contract

SSLR's contract — read-only connection, no server-side config, xmin tracking for inserts/updates, PK-hash divide-and-conquer for deletes, throttled chunked execution, replication cursor in target DB — is a known-good runtime pattern for one-way sync. We are not the first to need this contract; we should not reinvent it. The Step #2 reverse-engineering pass should produce a behavioral spec card precise enough to re-implement in TypeScript without consulting SSLR's source again.

5.6 Smart form-fill beats naive name matching

FormFilla's runtime behavior — multi-signal scoring, structural block detection, bipartite matching — is the right contract for our browser-extension adapter. Naive field-name matching will break on the first hotel that uses non-standard field names (which is most hotels). The behavioral contract to reproduce: each candidate form field receives a confidence score against each profile field; the assignment is solved as an optimization problem; the receptionist sees the proposed mapping and confirms before values are written.

6. Conclusions and Recommendations for Step #2

Discovery identified five open-source repositories that collectively cover every runtime behavior the Guest Profiling System must reproduce. No proprietary PMS software was inspected; all five are open-source and publicly documented. None will be forked or copied; only their settled runtime behavior will be captured as stack-agnostic behavioral specification cards in Step #2.

The recommended order for Step #2 reverse-engineering, by descending value-to-effort ratio, is:

- SSLR first. Its runtime contract is the spec our adapter must match. Cloning and exercising it locally is fast (single Go binary) and yields the highest-value spec card.
- AWS Sample Hospitality Systems second. Cloning the repository, deploying its SAM template against a local Lambda emulator, and capturing exact request/response JSON shapes for the guest API endpoints.
- KevAlien CRM third. Reading the GuestState TypedDict and the state-machine transition table from source; the WhatsApp and LLM portions are skipped entirely.
- QloApps fourth. Cloning the repository, inspecting its MySQL schema (the README does not document it), and capturing the runtime contract of a hotel PMS guest record.
- FormFilla fifth. Installing the published extension into a test browser, observing its scoring against sample registration forms, and capturing the trigger model and confirmation flow.

Each Step #2 spec card will contain: inputs (shape, validation, error cases), outputs (shape, status codes, error responses), latency (median and p95 under representative load), ranking/dedup rules, edge cases, error behavior, and data shape — and no copied code.

The output of Step #2 is a single behavioral specification document that becomes the input to the Product-Manager planning step (Step #3) and the OpenSpec design step (Step #4).

One caveat that should be carried into Step #3: SSLR's license is not explicitly stated in its README. Before any reuse of code (not just behavior) is considered, the license file in the repository must be read and confirmed compatible with the project's intended license. Behavioral study is permitted regardless of license; code reuse is not.

7. Appendix — Comparison Matrix

The matrix below summarizes the five reference repositories against the behavioral surfaces they cover. A filled cell (●) means the repository is the primary behavioral reference for that surface; an empty cell (○) means it touches that surface but is not the primary reference.

Table 3: Reference repositories × behavioral surfaces

Repository	PMS source shape	Guest API contract	State machine	Read-only sync	Smart form-fill
QloApps	●	○	○	○	○
AWS Sample Hospitality	○	●	○	○	○
KevAlien CRM	○	○	●	○	○
erkkah/sslr	○	○	○	●	○
FormFilla	○	○	○	○	●

7.1 Source URLs

QloApps: github.com/qloapps/qloapps · Latest release 1.8.0 (2026-11-06).

AWS Sample Hospitality: github.com/aws-samples/sample-hospitality-systems · Serverless hospitality platform, AWS SAM template.

KevAlien CRM: github.com/KevAlien/CRM · LangGraph + Redis + WhatsApp + Ollama.

erkkah/sslr: github.com/erkkah/sslr · Single Go binary; read-only PostgreSQL replication.

FormFilla: github.com/AhmadHassan-BTed/FormFilla · TypeScript Chrome/Edge extension; 4.7k stars.